PORTFOLIO



Somewhere



BANner it!

이세종

Android Developer

snm40109@gmail.com

Somewhere

AI 여행 생성·수정과 지도 시각화를 통해 효율적인 여행 계획을 돕는 **여행 계획·정산 앱**

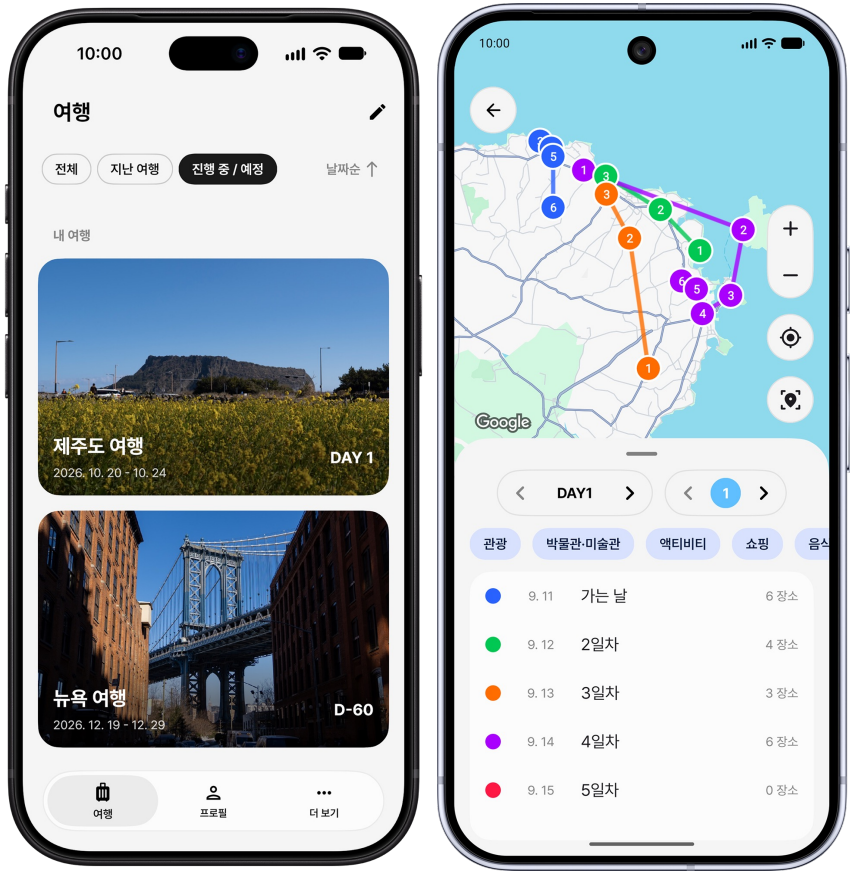
1인 개발 Android · iOS 앱

기간 2023.04 – 현재 (운영 중)

성과 다운로드 수 800+ | MAU 550+ | DAU 50+

기술 스택

App	`Kotlin` `Jetpack Compose` `KMP` `CMP` `MVVM` `Coroutine`
Data	`Firebase Firestore` `Firebase Storage` `Preferences Datastore`
Map	`Google Maps` `Google Maps Places`
etc	`Firebase Authentication` `Firebase Functions` `OpenAI API`



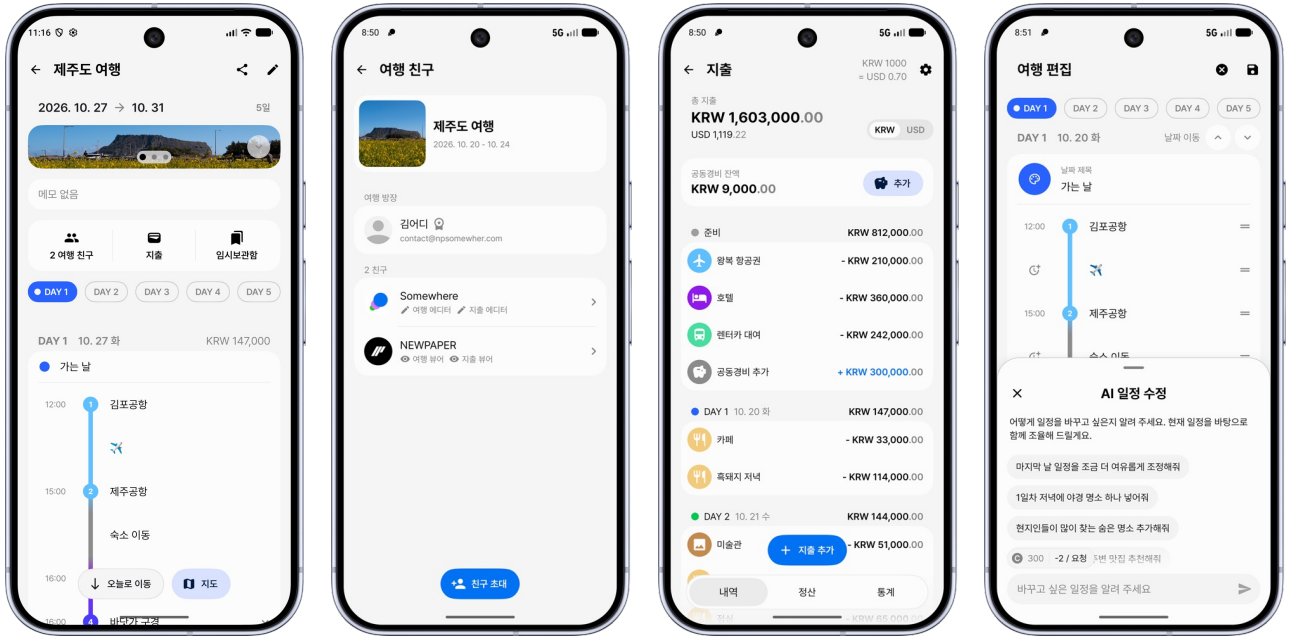
Download https://npsomewhere.com/download

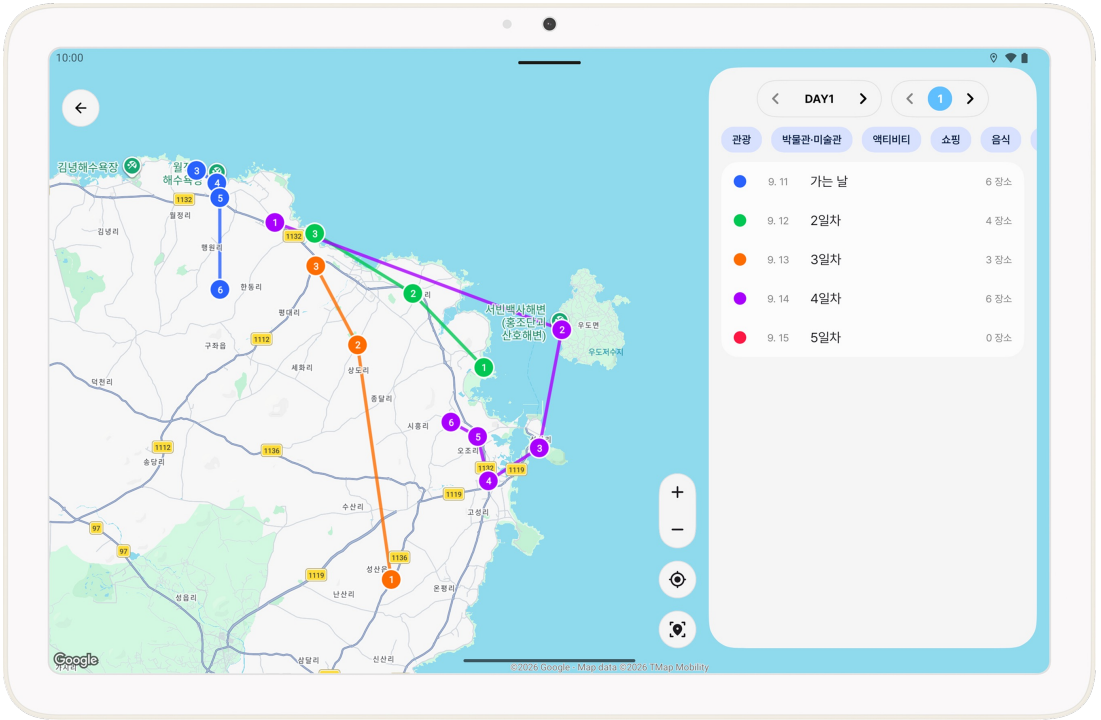
Release Note https://npsomewhere.com/whats-new/

Google Play, App Store

Somewhere

프로젝트 소개





- 여행 일정을 지도상에 시각화하기 위해 Google Maps를 활용해 **마커 및 라인 경로를 동적으로 표시**하는 기능 구현
- Firebase Firestore, Storage 기반으로 **친구 초대 · 공동 편집 기능** 구현
- 앱 전체 UI를 Compose로 구성하고, **TalkBack 접근성** 및 **대화면 UI 레이아웃**을 별도로 설계해 다양한 사용 환경에 대응
- LLM API를 통한 사용자 취향 맞춤 **일정 자동 생성** 및 **일정 동적 수정 · 재조정 기능** 구현

앱 진입 속도 개선 및 데이터 갱신 문제 해결

- **문제점**

  기존 Splash 화면에서의 과도한 데이터 로딩으로 인한 초기 진입 지연

  데이터 요청 로직의 한계로 앱 실행 중 데이터 재갱신 불가능

- **해결**

  여행 데이터 로딩 시점을 Splash에서 메인 화면으로 옮겨, 메인 화면 진입 시 데이터를 로딩할 수 있도록 최적화

  Baseline Profile 적용으로 JIT(Just-in-Time) 컴파일 비용을 최소화하여 Cold Start 시간 단축

  Local Cache 적용으로 사용자 정보를 즉시 조회하여 네트워크 응답 대기 시간 제거

- **결과**

  앱 진입 시간 약 92% 단축으로 쾌적한 사용자 경험 제공 (3,640ms → 279ms)

  앱 내에서 데이터 재갱신으로 유연한 데이터 동기화 구조 확보

	TTID (ms)
개선 전	3,640
작업 최소화 / Baseline Profile	2,770
작업 최소화 / Baseline Profile / Local Cache	279

SM-S908N (API 36) / Stable Wi-Fi

TTID(Time to Initial Display): 앱 시작부터 Splash 끝날 때까지

아키텍처 최적화

- **문제점**

  ViewModel 비대화: 단일 ViewModel에 여러 화면의 로직이 집중되어 유지보수성과 코드 가독성 저하

  강한 의존성: UI 계층이 데이터 구현체에 직접 의존하여 DB/API 변경 시 시스템 전반의 수정 필요

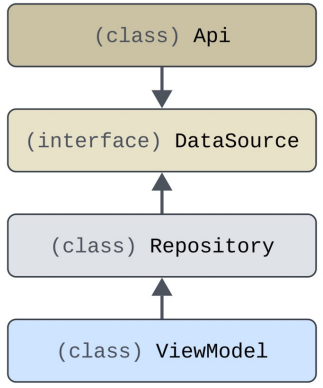
- **해결**

  - 각 화면의 역할에 맞춰 **ViewModel 분리**로 책임 분산
  - **UDF** 및 **중앙 집중식 상태 관리** 체계 구축
  - **DI** 및 **DIP** 적용을 통한 계층 간 결합도 해소

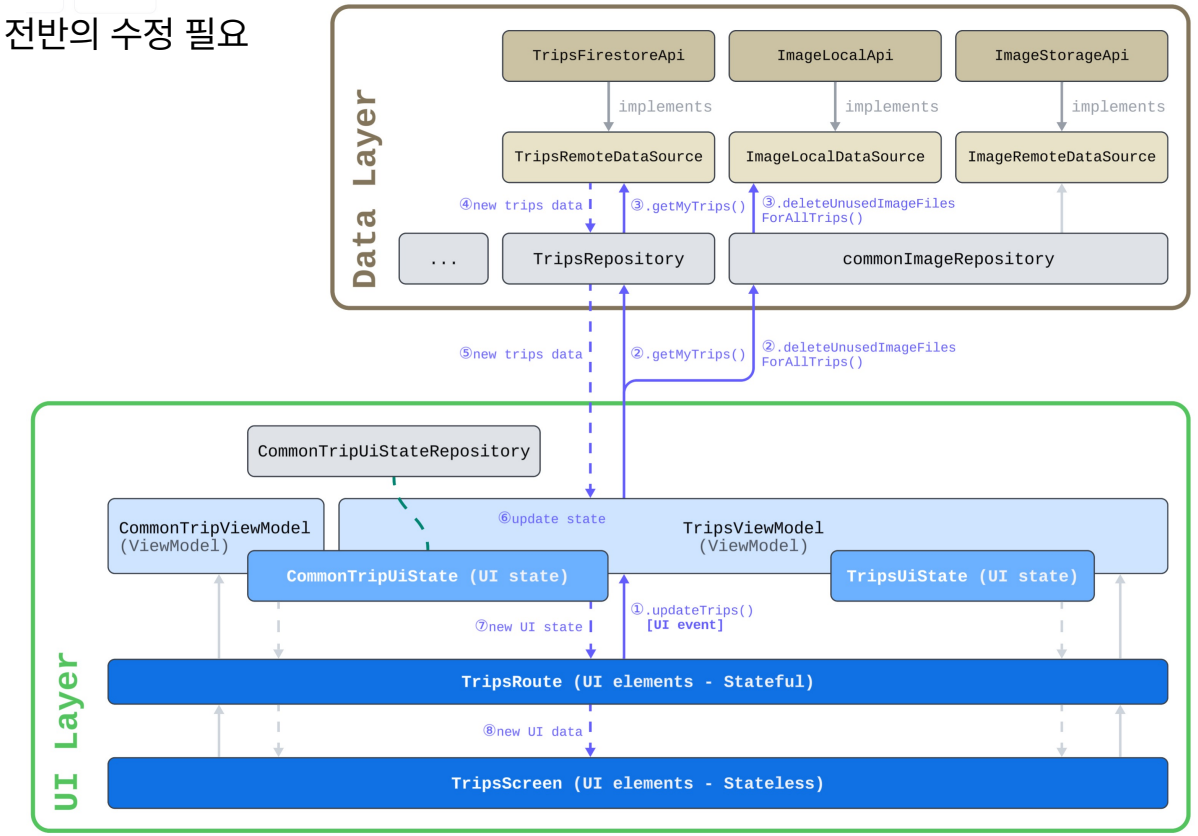
- **결과**

  유지보수 효율: 계층 간 결합도 완화 및 기능별 분리로 코드 수정 시 Side Effect 최소화

  구조적 유연성: DIP 기반 설계로 데이터 소스 변화에 영향받지 않는 견고한 아키텍처 구축



DI, DIP



UDF, 상태 관리

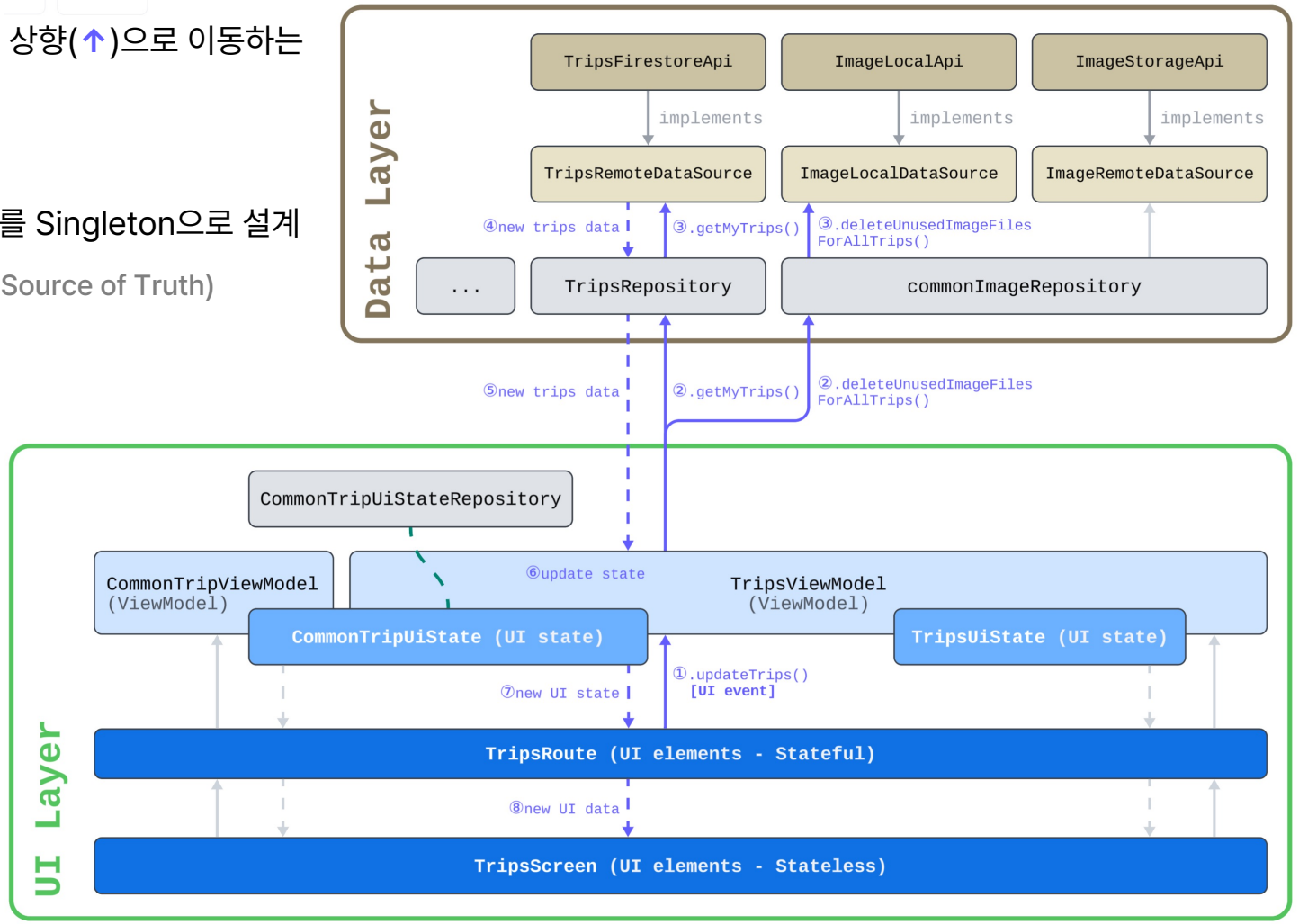
아키텍처 최적화 - UDF 및 상태 관리 적용 (Unidirectional Data Flow)

- **단방향 데이터 흐름 (UDF):** UI State는 하향(↓), UI Event는 상향(↑)으로 이동하는 단방향 데이터 흐름 구조 → 상태 변화의 예측 가능성 확보

- **중앙 집중식 상태 관리:** `CommonTripUiStateRepository`를 Singleton으로 설계
  → 여러 ViewModel 간 공유 데이터의 일관성(SSoT - Single Source of Truth) 및 무결성 유지

- **관심사 분리(SoC - Separation of Concerns):**
  Model · View · ViewModel 역할 분리
  → 각 계층의 독립성 및 재사용성 극대화

- **안정적인 UI 업데이트:** Data Layer의 변경이 ViewModel을 거쳐 UI에 안전하게 발행되는 구조 구축
  → 복잡한 비즈니스 로직에도 안정성 유지



대화면 UI/UX 적용

- **문제점**

  하나의 화면에 List/Detail 구조의 2-Pane UI를 구현하는 과정에서 Navigation 처리의 복잡도가 크게 증가 및 네비게이션 관련 버그 발생

  Google의 Adaptive Layouts를 활용하려 했지만 커스터마이징(너비 비율, 애니메이션)에 제약이 있어 직접 구현을 선택

- **해결**

  단일 NavHost 내에서 **조건부 레이아웃 분기 구조**로 설계

  화면 크기 변화에 따라 1-Pane ↔ 2-Pane 전환 시에도

  백스택 계층을 동적으로 재구성하여, Navigation 흐름을

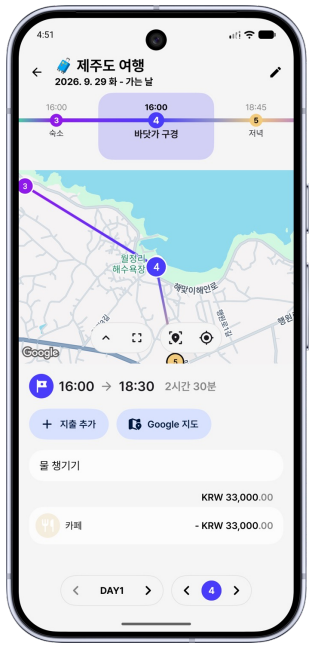
  안정적으로 유지

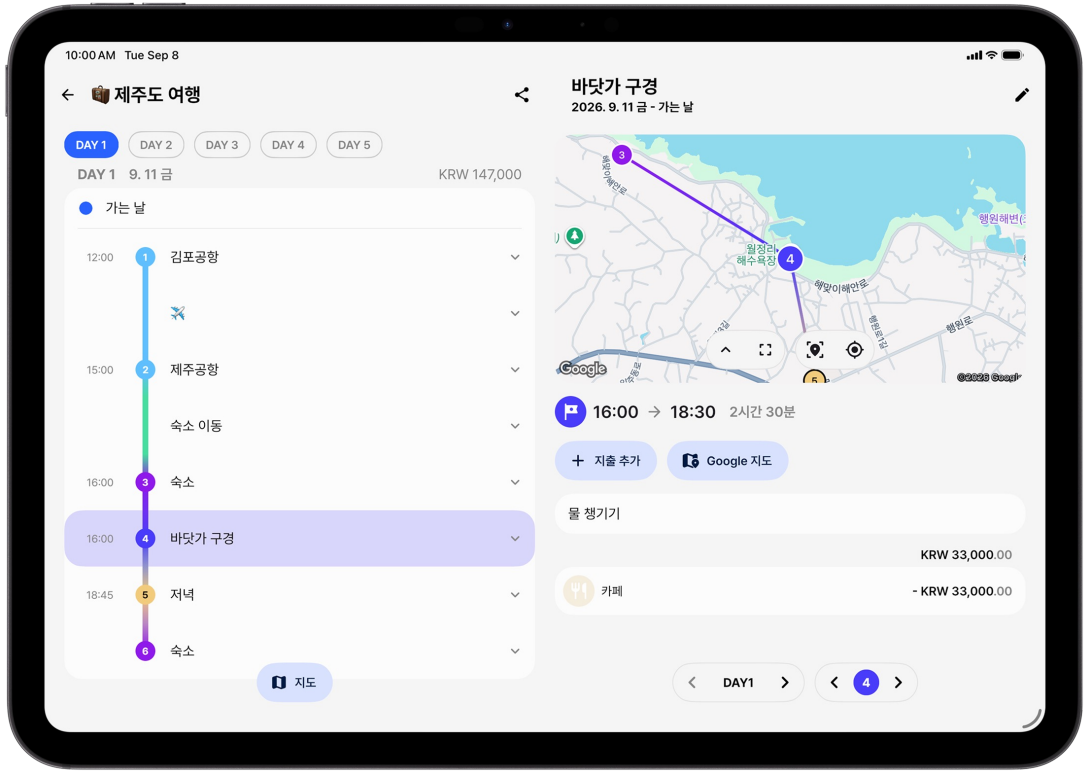
- **결과**

  직접 2-Pane 구조를 구현하며

  화면 크기에 따른 UI/UX 최적화를 설계했고,

  다양한 디바이스에서도 일관되게 작동하는 구조 설계





Android Native → KMP 마이그레이션

- **문제점**

  기존에는 Android 버전만 지원하여 iOS 사용자와 함께 여행을 갈 경우 일정을 공유할 수 없었음

  이러한 플랫폼 제약은 서비스 확장과 다양한 유저층을 유입시키는 데 큰 한계로 대두됨

- **해결**

  - 기존에 사용하던 Kotlin과 Jetpack Compose의 UI 코드 및 데이터 로직 대부분을 그대로 재사용할 수 있다는 장점에 주목하여

  KMP(Kotlin Multiplatform)를 마이그레이션 기술로 채택

  - Claude, Codex, Antigravity를 적극 활용하여 Android Native에서 KMP로의 최적화된 전환을 유도하는 마이그레이션 프롬프트 설계

  - AI 에이전트를 통해 phase0.md부터 phase8.md까지의 **상세 단계별 로드맵**을 수립하고, 각 페이즈의 기술 전환 작업을 AI와 긴밀히 협업·주도하며

  마이그레이션 프로세스를 획기적으로 가속화

- **결과**

  약 3주 만에 전환을 완료하고 iOS 신규 출시 (2026.06), iOS 출시 후 4개월간 **iOS 다운로드 600+**

  단일 코드베이스로 Android · iOS 주간 동시 배포 체계 확보



장애 감지 및 원격 차단 체계 구축

- **문제점:** 1인 운영이라 사용자 신고 없이는 오류를 알기 어렵고,
  서버 점검이나 긴급 대응 시 앱의 데이터 쓰기를 즉시 중단할 수단이 없어 데이터 정합성 훼손 위험 존재
- **해결**
  - Sentry handled error를 기능·원인별로 분류하고 R8 매핑을 업로드해 릴리스 스택트레이스 복원
  - 어드민 기반 Read-only 점검 모드를 클라이언트·Rules·Functions 3중으로 적용(fail-closed)
  - 최소 지원 버전 기반 강제 업데이트로 문제 버전의 사용 차단
- **결과**
  로그인 에러, iOS 트랜잭션 크래시 등 **실기기 크래시를 추적해 수정**
  점검·장애 상황에서 앱 배포 없이 원격으로 쓰기 차단 및 업데이트 유도 가능

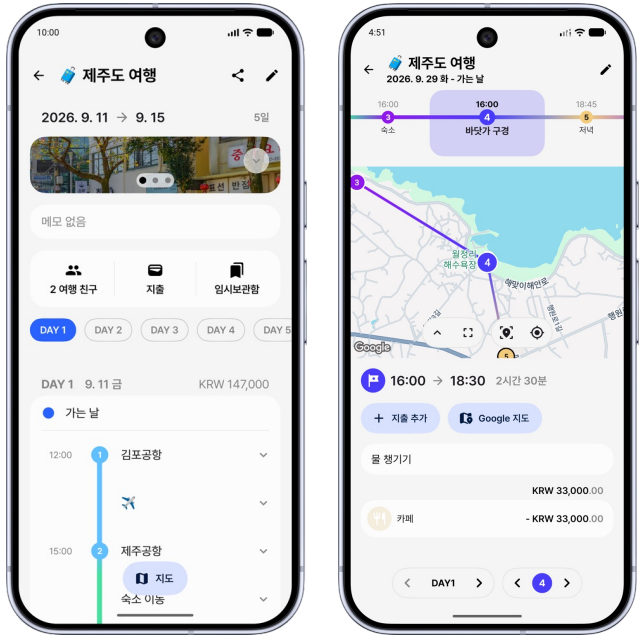
실패에 강한 AI 요청 처리

- **문제점:** AI 요청 중 앱이 종료되거나 네트워크가 끊기면 결과가 사라지고 재요청 시 중복 처리 위험
- **해결:** 요청을 호출 전에 로컬에 영속화하고 동일 요청 ID로만 재시도(멱등성), 응답도 적용 전에 저장
- **결과:** **앱 종료나 네트워크 실패 후에도 안전하게 복구**

Somewhere

UI/UX 개선

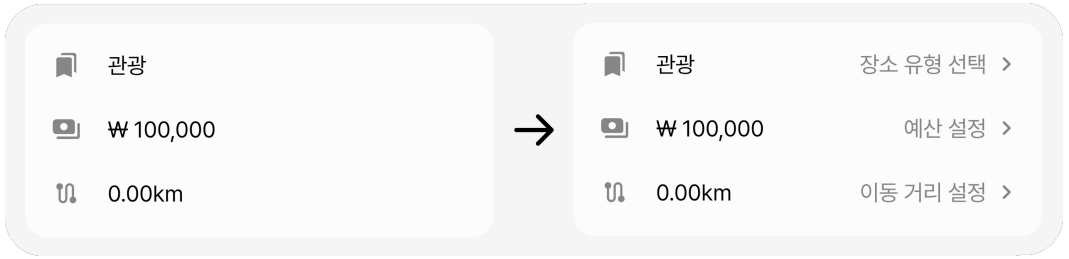
- **문제점:** 앱 사용 과정에서 Trip → Date → Spot의 세 단계 화면을 거쳐야만
  Spot 화면에 도달할 수 있는 비효율 구조가 존재했음
  또한 Trip, Date 화면이 유사하여 사용자가 혼동을 겪는 문제가 존재했음
- **해결:** Trip과 Date 화면을 하나의 화면으로 통합하여, Spot 화면까지의 **접근단계를 두 단계로 축소** (Trip → Spot)
- **결과:** 화면 이동 과정을 간소화함으로써 **사용자 경험의 직관성과 효율성을 높임**



Trip → Spot

사용자 피드백 기반 UI/UX 개선

- **문제점:** 앱을 테스트해 보는 과정에서 "무엇을 눌러야 할지 모르겠다"는 피드백을 받고 UI/UX를 개선
  기존에는 카드 형태의 UI에 아이콘과 텍스트만 있어 상호작용 가능한 요소로 인식되기 어려웠음
- **해결:** 우측에 회색 힌트 텍스트 및 아이콘(>)을 추가하여 상호작용 가능한 요소로 인지할 수 있도록 개선
- **결과:** UI/UX 구성에 있어 **사용자 관점에서의 직관성과 접근성을 고려**하며,
  실질적인 개선 경험을 쌓을 수 있었음
  사용자 피드백을 통해 직관적인 UI/UX의 구성의 중요성을 체감



BANner it!

YOLO, OCR, LLM을 활용한 **불법 현수막 신고 앱**

팀 프로젝트 (4인) · Android 앱

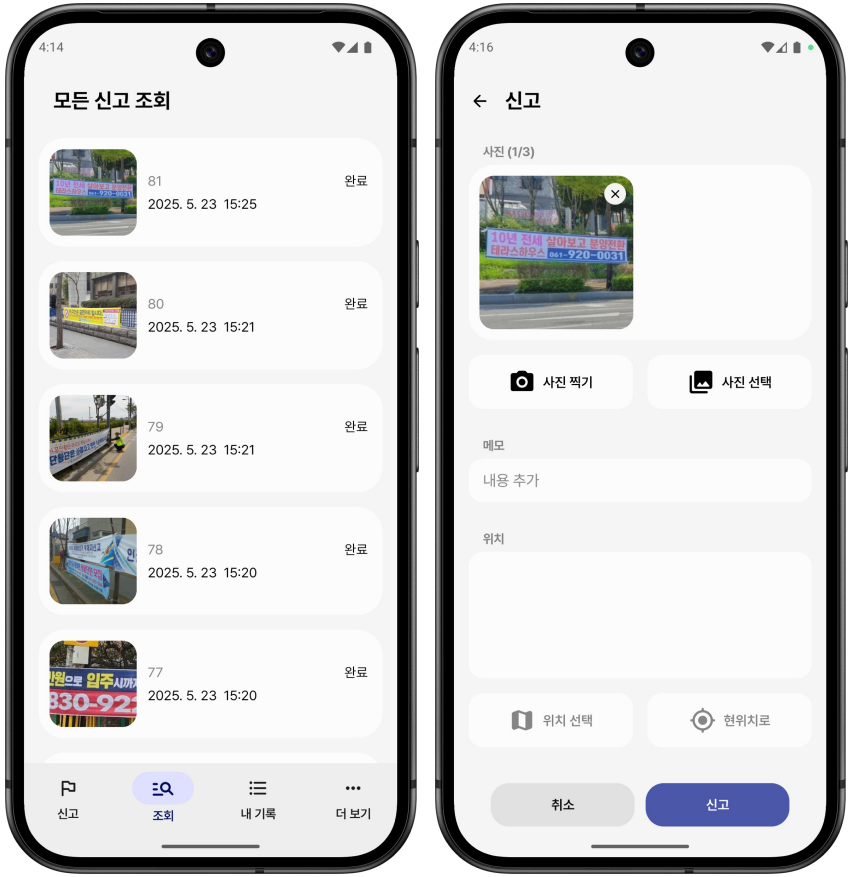
기간 2025.03 – 06

성과 제 19회 세종대학교 창의설계경진대회 우수상 2025.06

한국데이터사이언스학회 춘계종합학술대회 장려 논문상 2025.06

역할

- Android 앱 화면 설계 및 앱의 모든 기능 구현
- 원활한 협업 환경 구축을 위해 프로젝트 문서 정립과 작성을 주도
- LLM 학습 데이터 구성과 파인튜닝을 직접 주도하며, 모델 성능을 기존 대비 26%p 향상
- 기여도: 전체 30% – Android (100%), LLM 파인튜닝 (40%), 팀 리딩 (30%)

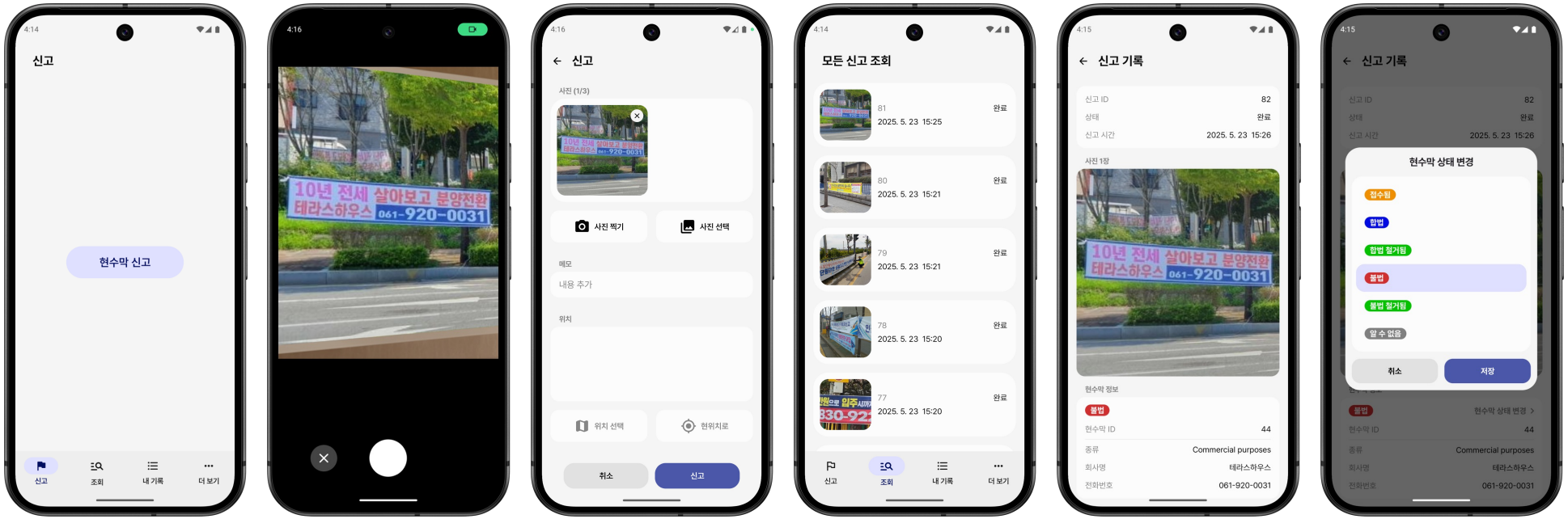


Android 기술 스택

`Kotlin` `Jetpack Compose` `MVVM` `Coroutine` `Hilt` `Retrofit` `CameraX`

GitHub https://github.com/FITIFITBANnerit

BANner it!



- CameraX를 사용하여 **실시간 프리뷰** 및 **사진 촬영 기능**을 구현하고, Bitmap/Exif 기반의 이미지 회전·압축 로직을 구축
- Retrofit을 활용해 REST API 기반 백엔드 서버와 통신하여 **이미지 업로드 및 분석 결과 조회 기능 구현**
- 사용자가 앱에서 현수막 신고를 하면, YOLO를 사용해 **현수막 객체를 탐지**하고, PaddleOCR로 해당 영역의 **텍스트를 추출**
- 추출된 텍스트는 LLM(HyperCLOVA X)에 전달되어 **불법 여부를 판단하고, 상호명과 전화번호 추출**

API 규칙 수립을 통한 협업 효율 개선

- **문제점**

  Android와 백엔드 간에 명확한 API 규칙이 없어, API를 추가/수정할 때마다

  요청/응답 구조, 변수명, 데이터 포맷에 대한 불필요한 의사소통이 반복적으로 발생

- **해결**

  API 규칙을 정의한 문서를 작성하여 JSON 네이밍 규칙(snake_case), 날짜/시간 포맷(ISO 8601, UTC), 위치 데이터 구조,

  공통 응답 및 오류 포맷, enum 코드값, JWT 인증 방식 등을 포함한 **API 규칙 문서를 주도적으로 작성하고 표준화함**

  이를 기준으로 Android와 백엔드 간 협업을 진행하도록 제안하고 주도함

  API 규칙 문서: https://future-reptile-53f.notion.site/API-1406f531c61c83d38b37811b638caf50

- **결과**

  API 개발 및 수정 시 **커뮤니케이션 비용이 크게 감소**

  Android와 백엔드 모두 예측 가능한 구조로 개발할 수 있어 개발 속도와 코드 일관성 향상

  협업 관점에서의 문제 해결 경험과 기술적 커뮤니케이션 역량을 강화할 수 있었음

하드웨어 센서 대응 이미지 프로세싱

- **문제점:** 사용자가 기기를 세로 또는 가로로 촬영할 때, 이미지가 회전되어 저장되는 문제 발생

  기기의 하드웨어 센서 방향과 사용자의 촬영방향이 일치하지 않아 발생하는 문제
- **해결:** ExifInterface를 활용해 이미지의 회전 값을 분석하고, Matrix 연산을 통해 정방향으로 **자동 보정하는 로직 적용**
- **결과:** 사용자가 기기를 어떤 방향(세로, 가로)으로 들고 촬영하더라도 항상 의도한 방향의 이미지를 얻을 수 있음

```
val bitmap = BitmapFactory.decodeFile(photoFile.absolutePath) ?: return
val matrix = Matrix().apply { postRotate(rotation.toFloat()) }
val rotatedBitmap = Bitmap.createBitmap(bitmap, 0, 0, bitmap.width, bitmap.height, matrix, true)
```

CameraPreviewViewModel.kt

Bitmap 객체로 변환 / Matrix 객체 생성 및 회전 설정 / 회전된 Bitmap 생성

이미지 처리 및 메모리 최적화

- **문제점:** 고해상도 원본 이미지를 그대로 처리·업로드하여 메모리 사용량과 전송 용량 부담 발생
- **해결:** 이미지 가공 직후 사용이 끝난 Bitmap을 recycle()로 즉시 해제하고, Bitmap.compress()로 업로드 파일 크기 최적화
- **결과:** 이미지 처리 과정의 메모리 사용량을 줄여 **앱 안정성을 높이고, 데이터 전송 및 저장 공간 효율 개선**